

Inter-Integrated Circuit on the MUNSTAR-1 Satellite: Development and Implementation



Sam Ruttgaizer

202312489

August 23, 2024

Inter-Integrated Circuit on the MUNSTAR-1 Satellite: Development and Implementation

Prepared for Mr. William Griffin
Co-operative Education
Faculty of Engineering and Applied Sciences
Memorial University of Newfoundland and Labrador

Prepared by Sam Ruttgaizer
MUNSTAR-1 Satellite Mechanical/Mechatronics Engineering Student
C-Core
MUN Student Number: 202312489

August 23, 2024

Executive Summary

The MUNSTAR-1, a CubeSat in development by a student team at Memorial University and funded by the Canadian Space Agency CUBICS initiative, C-Core, and Solace Power, is a significant step forward in furthering interest in space in Newfoundland and Labrador. Intending to collect data from our oceans, it also serves as a gateway for students to get hands-on with learning and developing technology built for space.

A vital aspect of any satellite is the onboard communication protocols, transferring essential information with minimal errors at high speed between the subsystems. Without reliable communication with the onboard computer, it cannot confidently make the correct positioning, power consumption, and data collection decisions.

Inter-Integrated Circuit is one of the main communication methods taken advantage of on MUNSTAR-1, and it is responsible for commands and responses to and from the Electrical Power System, Global Positioning System, and onboard computer. Using only two lines for the clock and data and the ability to address multiple devices on a single bus, Inter-Integrated Circuit gives ample flexibility without unnecessary complexity, making it perfect for use on the satellite.

The software developed for Inter-Integrated Circuit communication onboard MUNSTAR-1 is robust and error-limited. Through extensive testing, communication is successful between the computer and subsystems, allowing complete control over the subsystems through software.

Table of Contents

1 Introduction	1
2 Communication Overview	3
2.1 Communication Framework	3
2.2 Abstraction Layers	4
2.3 Development Tools	5
3 Inter-Integrated Circuit	6
3.1 I2C Message Frames	6
3.2 Development	7
3.2.1 Initialization	7
3.2.2 I2C Read	8
3.2.3 I2C Write	9
4 EPS Implementation	11
4.1 Command and Response Messages	11
4.2 Message Functionality	13
4.3 Message Handling	14
4.3.1 Standard Messages	14
4.3.2 Responses with Telemetry	15
5 GPS Implementation	16
5.1 Command and Response Messages	16

5.1.1 Telemetry Commands	17
5.2 Message Handling	19
6 Testing	20
6.1 Logic Analyzer Testing	21
6.2 CuteCom Testing	23
7 Conclusion	25
8 References	26

List of Figures

Figure 1: Diagram of communication protocols used on MUNSTAR-1	4
Figure 2: The layers of programming on MUNSTAR-1	5
Figure 3: Message format of an I2C transmission.....	6
Figure 4: Structure of messages between the OBC and EPS	11
Figure 5: ZedBoard (left) connected to EPS (middle), powered by EGSE (right)	20
Figure 6: Logic analyzer (left) tapping I2C lines connecting ZedBoard and GPS	21
Figure 7: SCL and SDA lines recorded in Logic 2 software	22
Figure 8: SupMCU LED on after command	22
Figure 9: Output of GPS "SUP:TEL? 0,ASCII\n" in CuteCom	23
Figure 10: Output of the EPS Get System Status command in CuteCom	24

List of Tables

Table 1: Descriptions of functions currently used on MUNSTAR-1	13
Table 2: The CC and RC of each command with the expected write and read size.....	14
Table 3: Description of commands used by the GPS	17
Table 4: All possible SupMCU telemetry commands	18
Table 5: All possible GPS telemetry commands	19

List of Acronyms

BID - Board Identifier

CAN - Controller Area Network

CC - Command Code

CSA - Canadian Space Agency

EGSE – Electrical Ground Support Equipment

EPS - Electrical Power System

GPS - Global Positioning System

HAL - Hardware Abstraction Level

I2C - Inter-Integrated Circuit

ICEPS2 - ISISpace Compact Electrical Power System version 2

IVID - Interface Version Identifier

OBC - Onboard Computer

RC - Response Code

RS-485 - Recommended Standard #485

SCL - Serial Clock

SDA - Serial Data

SoC - System on a Chip

STAT - Response Status Information

STID - System Type Identifier

SupMCU - Supervisor Microcontroller

UART - Universal Asynchronous Receiver / Transmitter

XSA - Xilinx Support Archive

1 Introduction

CUBICS, an initiative created by the Canadian Space Agency (CSA), incentivizes universities nationwide to develop nanosatellites known as CubeSats [1]. A CubeSat is a small satellite with a standardized form factor of units, with one unit (1U) equivalent to 10x10x10cm. CubeSats perform various earth-sensing tasks and are primarily developed by students with the help of university professors and industry professionals.

MUNSTAR-1 is Newfoundland and Labrador's second Earth observation satellite and is being developed by students at Memorial University in collaboration with C-Core and Solace Power. With C-Core being the leading partner of the MUNSTAR-1 project, they have provided work terms to many engineering students to further the project's progress, along with providing expertise in remote sensing technologies. They were also the leading partner of Killick-1, Newfoundland's first satellite, so they have experience in all design stages.

While being relatively small, the MUNSTAR-1 hosts many complex subsystems required for smooth operation and achieving its goal of measuring sea conditions via reflected GPS waves. These subsystems include the payload, electrical power system (EPS), communications, GPS, and Attitude Determination and Control System.

Reliable communication with the onboard computer (OBC) is integral to these subsystems. The data received from the subsystems allows the OBC to choose the optimal course of action for satellite function. That means bulletproof communication programs must be developed to send and receive data. While the idea of communication may seem straightforward, there are a multitude of communication

protocols that all have different uses and standards. This report will serve as a resource and progress report to future developers, diving into the usage and testing of the Inter-Integrated Circuit (I2C) protocol onboard the MUNSTAR-1 satellite. The Communication Overview section will cover satellite communication, the requirements, the development plan, and the tools used. The following section details the inner workings of the I2C protocol, how it works, why it is used, and the programs developed to use the protocol. The EPS and GPS sections describe using I2C to communicate with the OBC and testing completed to ensure functionality.

2 Communication Overview

The MUNSTAR-1 uses serial communication protocols to allow the OBC to send and receive data to and from the other subsystems. Different protocols are required based on the specifications of the subsystem, the importance of the message being transmitted, and hardware restrictions. The satellite uses multiple protocols, including the previously mentioned I2C, controller area network (CAN), universal asynchronous receiver/transmitter (UART), and recommended standard #485 (RS-485). These protocols were developed with the ZedBoard development board, which uses the Xilinx Zynq-7000 system on a chip (SoC). This is the same SoC used by the OBC, giving the ability to test and develop software without the risk of damaging the expensive, flight-ready OBC.

2.1 Communication Framework

While the GPS and EPS share an I2C bus, all other subsystems have isolated lines for communication with the OBC. Figure 1 depicts how all subsystems on the satellite are linked to the OBC. Xilinx provides libraries for each communication protocol, providing drivers to help developers configure functions to send and receive data. All launch-ready software is written in the programming language C.

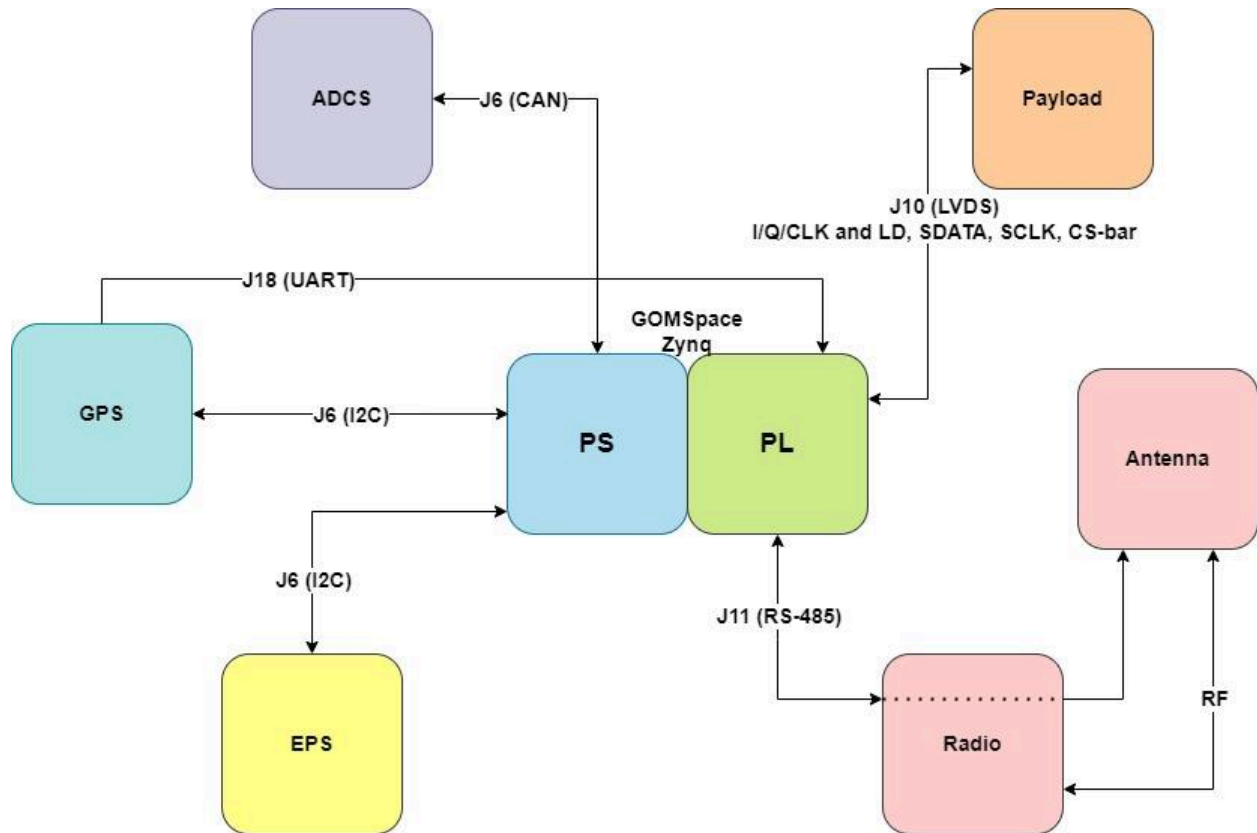


Figure 1: Diagram of communication protocols used on MUNSTAR-1

Source: Primary

2.2 Abstraction Layers

To link the Xilinx drivers designed for ZedBoard and the Nanomind to the subsystems that use the communication protocols, there must be a layer of abstraction between the base Xilinx functions and the subsystems. The hardware abstraction layer (HAL) accomplishes this by creating wrappers around the Xilinx functions, extending functionality, and tailoring them to the specific use case of the satellite. A wrapper can be a function that wraps around another function for compatibility or to add features [2]. These wrappers add improved error handling, which converts the less legible Xilinx error codes into usable `Status_t` statuses, a status standardized throughout all protocols on the satellite. By wrapping these functions, additional assertions are also

carried out, ensuring the code behaves correctly and increasing reliability and error handling. The client, which in this case is the subsystem module (EPS, GPS, etc.), will access the HAL functions to send specific messages using the specified communication protocol. The wrapper simplifies required arguments that must be passed into the Xilinx functions, drastically reducing the code's complexity, so the client only needs to pass minimal information to the HAL functions. This essential layer is a low-level bridge for the subsystem and state machine programs built on top to access the Xilinx drivers below. Figure 2 overviews these layers, from lowest to highest level.

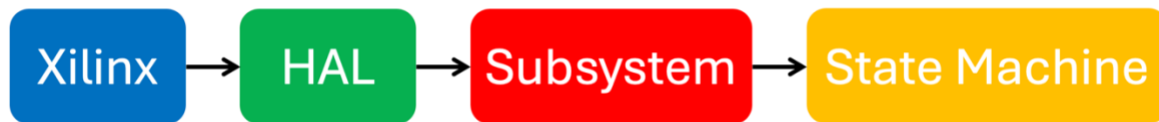


Figure 2: The layers of programming on MUNSTAR-1

Source: Primary

2.3 Development Tools

The Xilinx Vivado and Vitis software is used to develop the Zynq SoC on MUNSTAR-1. Vivado is a software development suite tailored to Zynq SoCs. First, a hardware design is created using the Vivado software, which creates a Xilinx support archive (XSA) file. The Vitis integrated development environment can create a platform with this XSA file, allowing developers to write applications and scripts for the specific hardware design. This hardware design includes input/output and pin mapping to enable the ZedBoard to use the particular hardware design. The software that can be developed in Vitis is dependant on how the hardware has been designed in Vivado.

3 Inter-Integrated Circuit

I2C is a standard serial communication method for interfacing with sensors, displays, and other electronics. I2C only uses two lines: one for data (SDA) and one for clock (SCL) [3]. The basic principle is that there is a single (or multiple) controller(s) and multiple targets on the I2C bus.

3.1 I2C Message Frames

Many targets (up to 127 with 7-bit addressing) can tap into the lines but can only access the line when the controller sends the target's specific 7-bit address [3]. An I2C message begins with a start condition: the SDA line gets pulled low before the SCL line does. Once this occurs, a transfer is initiated by the controller, sending the address of the target device along with a read or write bit, as seen in Figure 3. After this happens, the controller requests an acknowledgment bit from the target, confirming if the target has received the address. If acknowledged, the controller continues to read from or write to the target in one-byte intervals until a stop condition is initiated.

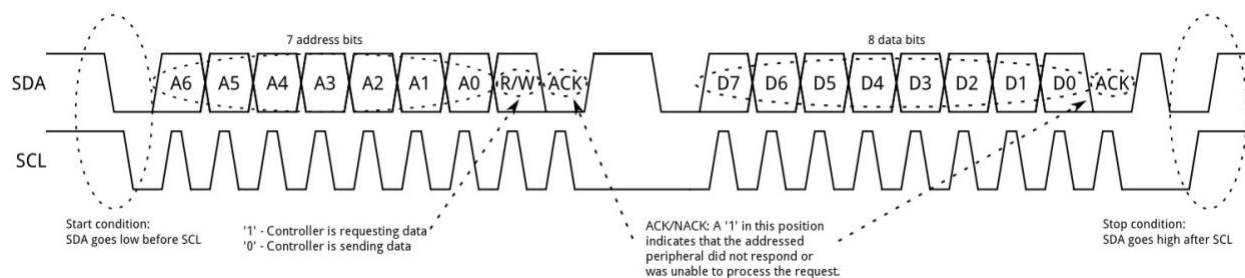


Figure 3: Message format of an I2C transmission

Source: [4]

The clock on an I2C bus has multiple speed standards that, depending on the devices on the bus, can be set. The most common are standard and fast modes, with clock speeds 100KHz and 400KHz, respectively. The faster the clock, the quicker the data speed, although moving to speeds beyond fast mode may have extra requirements and different implementations of I2C to allow usage of these modes [3].

3.2 Development

The Xilinx iicps module includes functions for both polling and interrupt-driven I2C communication. The MUNSTAR-1 will use polled I2C to communicate between the OBC, EPS, and GPS. The I2C functions that were developed were based on the read and write functions used on the previous Killick satellite.

3.2.1 Initialization

To read and write using this module, the I2C driver must first be initialized. By referencing the example programs that Xilinx provides, along with using assertion functionality, the initialization process is as follows:

- First, a structure variable of the config structure that holds variables of all information needed to configure the I2C controller (ZedBoard) is recorded using the `XIicPs_LookupConfig` function [5], which returns a pointer to configuration, or NULL if the base address passed into the function is not found.
 - Note: The I2C base address is a value generated in the Xilinx `xparameters.h` file based on the hardware design used.
- `XIicPs_CfgInitialize` is called [5], initializing the specific device whose config must be passed as an argument into the function.

- Next, `XIicPs_SelfTest` is called [5], running a self-test that involves testing and then resetting the device.
- Finally, the clock speed is set using `XIicPs_SetSClk` [5]. This function allows a frequency in Hz to be passed as an argument to set the controller's clock. It will be set to I2C fast mode (400KHz).

3.2.2 I2C Read

The `i2c_read` function wraps around the `XIicPs_MasterRecvPolled` function defined in the `xiicps` module [5]. The Xilinx function has four parameters:

- `XIicPs * InstancePtr`: Pointer to the struct variable that holds all configuration and initialization information
- `u8 * MsgPtr`: Pointer to the buffer that will hold the data to be read from the target
- `s32 ByteCount`: Amount of bytes that will be read
- `u16 SlaveAddr`: 7-bit address of the target to read from

The `i2c_read` function itself has three parameters:

- `const I2C_Address_t address`: 7-bit address of the target to be read from, all valid target addresses are stored in an enumeration with given type `I2C_Address_t`, for example, passing `GPS_I2C_ADDRESS` (a constant in the enumeration that holds the GPS address 0x51) as an argument to this function means this value will be used as the address to the device to be read from. This ensures no invalid address can be passed into this function and improves readability compared to using the hex values.

- `Vec_Ref_U8 read_data_ref`: A reference to a vector with an unsigned 8-bit type must be passed as the second argument to this function. This will be an empty vector with at least the minimum required read length or larger. This is where the data read from the target will be stored. Mutator methods will be used to fill this vector with the read data from the target.
 - Note: The vectors mentioned are proprietary to MUNSTAR-1, as no vectors exist in the standard C library. Essentially, they work as dynamic arrays, able to resize after inserting and deleting elements.
- `U32 read_len`: Number of bytes to be read from the target.

When `i2c_read` is called, it is first asserted that the bus is not busy (a transfer is currently taking place). Next, it asserts that the `read_len` is of the correct size (greater than 0, less than the maximum size, and less than or equal to the capacity of the reference vector). The Xilinx function is then called, and its return value is held in a status variable. Once the function has been called, and because you cannot pass a vector into the function as discussed above, the values stored in the `read_buffer` array are copied into the vector by looping the `vec_set` vector mutator until complete. Finally, a switch statement is used to change the Xilinx status into a `Status_t` status, which is then returned.

3.2.3 I2C Write

`i2c_write` wraps around the Xilinx function `XIicPs_MasterSendPolled` [5]. It also has four parameters:

- `XIicPs * InstancePtr`: Same as `XIicPs_MasterRecvPolled`

- `u8 * MsgPtr`: Pointer to the buffer that will hold the data to be sent to the target
- `s32 ByteCount`: Amount of bytes that will be written
- `u16 SlaveAddr`: 7-bit address of the target to write to

The `i2c_write` has two parameters:

- `const I2C_Address_t address`: same as `i2c_read`, this time the address of the target the controller is writing to.
- `Vec_Ref_U8 write_data_ref`: This vector will contain the data to be written to the target. This vector will be initialized as having a capacity containing the command to be sent.

When `i2c_write` is called, it is first asserted that the bus is not busy. The `write_len`, an internal variable that will be passed as the `ByteCount` argument in the Xilinx function, is set using the `vec_length` accessor within the vectors module. `vec_length` gets the number of items in the reference vector you pass to it. It is also asserted that `write_len` is less than the max transfer size. The data in the reference vector must now be copied to an array to be passed into the Xilinx function. This is done using `vec_get`, an accessor that will get the value at a specified index of a reference vector. With all the arguments ready, `XIicPs_MasterSendPolled` is called, and the same status switch case as `i2c_read` is used to return the status.

4 EPS Implementation

After the development of the HAL functions, the next step is the creation of the subsystem modules that use these functions, the EPS and GPS. With the base functions developed, they can now be configured to send and receive specific messages based on the format of the EPS. The model of the EPS being used on the MUNSTAR-1 is the ISISpace Compact Electrical Power System version 2 (ICEPS2) [6].

4.1 Command and Response Messages

To write to the EPS, a specific format of bytes must be sent after the device has acknowledged the address. This format, shown in Figure 4, is as follows and is the same for nearly all command messages, with some minor exceptions:

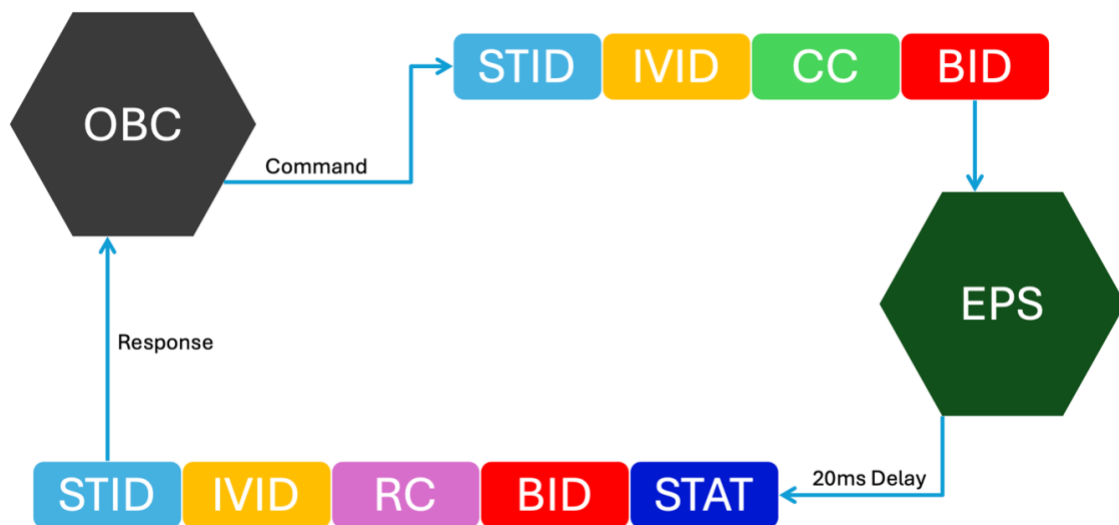


Figure 4: Structure of messages between the OBC and EPS

Source: Primary

STID (0x1A): The System Type Identifier (STID) is the first byte in every EPS transmission. It identifies what type of system is being transmitted to, as some systems have different components or are multi-board EPS. As the MUNSTAR-1 system is a single-board Power Integrated Unit, it has all of a multi-board system's functionality, including the Power Conditioning Unit, Power Battery Unit, and Power Distribution Unit, packaged together in a single board [6]. Because of this, we must use 0x1A as the STID byte.

IVID (0x07): Interface Version Identifier (IVID) is used to identify the software version running on the EPS. This allows the EPS to decode commands from different software versions depending on what IVID is passed. On MUNSTAR-1, version 7 runs, meaning the byte 0x07 is passed for this value.

CC: The Command Code (CC) is how the EPS is notified of what command it is being told to do. All commands for the EPS have a CC.

BID (0x01): Board Identifier (BID) is used to identify which board is being sent a command in multi-board systems. Because we only have one board, this will always be set to 0x01.

Depending on the command sent, reading from the EPS may be done to confirm the command was received and to read telemetry data if necessary. The EPS requires a 20-millisecond delay after writing to have a response ready to be read. A standard response is similar to the command, starting with the STID, IVID, then a response code (RC), BID, and Response Status Information (STAT).

The Response Code, specific to the command sent, indicates that the transmission was received. The response code will always be an increment of the CC. The STAT byte gives information based on the preceding command. This byte will notify the client whether the command was accepted (0x00) and, if it was rejected, for what reason.

If the command was to get telemetry, such as the Get System Status command, after the STAT byte, more information would be available to read, and the length in bytes would depend on the command sent.

4.2 Message Functionality

Table 1 briefly describes the functionality of all the currently implemented commands. Detailed descriptions can be found in [6].

Table 1: Descriptions of functions currently used on MUNSTAR-1

Source: Recreated from [6]

Command Name	Description
Nominal Mode	Nominal Mode allows access to all output buses, which is the default mode of the EPS when powered on.
Safety Mode	This mode turns off all non-automatically enabled output bus channels, rejecting any commands to these channels.
Get PIU Housekeeping Data	Fetches housekeeping data, including voltages, current, and power on the PIU board output channels and battery.
Get System Status	Fetches system information, including current mode, uptime, and number of resets.
Watchdog	Resets the watchdog timer, preventing a timeout and causing a reset.
System Reset	Performs software persuaded reset of the system.

4.3 Message Handling

Message handling is done within the EPS module. A vector is created for each command, holding the command's four bytes. The only exception is the System Reset command, which is five bytes, the additional byte being a reset key required to confirm the reset. Different receive vectors are defined for the different byte sizes that can be returned from the EPS. Table 2 contains information on all the functions currently developed for the EPS.

Table 2: The CC and RC of each command with the expected write and read size

Source: Recreated from [6]

Command Name	Command Code + Size	Response Code + Size
Nominal Mode	0x30, 4 bytes	0x31, 5 bytes
Safety Mode	0x32, 4 bytes	0x33, 5 bytes
Get PIU Housekeeping Data	0xA2, 4 bytes	0xA3, 116 bytes
Get System Status	0x40, 4 bytes	0x41, 36 bytes
Watchdog	0x06, 4 bytes	0x07, 5 bytes
System Reset	0xAA, 5 bytes	0xAB, 5 bytes

4.3.1 Standard Messages

With all vectors defined, the static function `eps_command` is also then defined. This EPS-internal function wraps the I2C read and write functions along with delays to create a specific command that takes a command vector as an argument, sends it through I2C, and reads the RC to check for errors. The RC of the command being sent is also passed as an argument. `eps_command` returns a status based on whether the I2C read

and write was successful and whether the RC read matches the RC passed to the function. With this command defined, functions for each EPS command were created. These functions require no arguments, generate the reference vector for the specific command, and pass it to `eps_command`, returning the status.

4.3.2 Responses with Telemetry

The commands Get PIU Housekeeping Data and Get System Status reply with additional information after the standard five-byte response, so `eps_command` cannot be used. The main problem with this is that this data has multiple different types, and copying it from a vector to a structure is more complicated.

I2C receives data in byte intervals, but because much of the telemetry data is in two-byte or larger form, the data must be spliced together after the transfer. The accessor `vec_get` was wrapped into a function that calls it twice for the first and second bytes of data. Because the data is received in little-endian form (the least significant byte comes first) [6], the second byte is shifted before the first, and the newly formed data is returned.

Now that there is a way to get different types of data out of the vector, both telemetry functions copy this data into a struct passed as a parameter to the function. The data structures include all the data types in the same order that they are received from the EPS. Both telemetry functions return statuses like the standard functions, but the struct is passed as an argument that contains all the data received from the command and can also be accessed by the client.

5 GPS Implementation

Communication with the GPS is similar in that it has a specific message structure and delay period. Although this structure is very different, communication happens in the same manner as with the EPS. The GPS model to be onboard MUNSTAR-1 is the Pumpkin GPSRM 1 GPS Receiver Module that uses a NovAtel OEM719 receiver [7].

Note: GPS also uses UART to send telemetry data, which is not discussed in this report.

5.1 Command and Response Messages

Commands for the GPSRM1 are issued in plain text form and must be ended with a newline character (\n) [7]. The format of the commands is as follows:

- The first three characters specify what part of the GPS you are communicating with, either the Supervisor or the GPS. The first three characters for Supervisor must be SUP; with GPS communication, use GPS. The Supervisor functionality consists of resetting the Supervisor Microcontroller (SupMCU), controlling the LED, Self-test, Clock functionality, and requesting telemetry data. The GPS functionality includes control over, power, reset, UART communication channel, and logging for the OEM719 receiver. After specifying either SUP or GPS, a colon follows.
- After the colon, the exact command is specified, followed by the argument selected for that command. See Table 3 for the commands developed for MUNSTAR-1; commands that have multiple arguments have them listed inside

<>. The Pumpkin SUPERNOVA Firmware Reference Manual includes tables for all commands for the Supervisor and GPS.

Table 3: Description of commands used by the GPS

Source: Recreated from [8]

Command	Description
SUP:TEL? 15,DATA\n	This command returns the temperature of the GPS in Kelvin in integer form.
SUP:RES NOW\n	Resets the SupMCU module.
GPS:RES <OFF ON>\n	Turn on and off the reset signal to the OEM719 receiver (the signal must be turned off automatically by the corresponding command).
GPS:PASS <OFF ON>\n	Controls UART passthrough channel to GPS receiver.
GPS:POW <OFF ON>\n	Turn on and off the OEM719 receiver.
SUP:LED ON\n (testing)	Changes the SupMCU LED to a solid color.
SUP:TEL? 0,ASCII\n (testing)	Returns firmware version of the SupMCU.

5.1.1 Telemetry Commands

Many telemetry commands are available for the supervisor and GPS, as indicated by adding “TEL?” after the colon. They differ from standard commands, requiring a number to specify the requested telemetry data. All telemetry has four possible arguments to be passed for each number:

- NAME: Sent in binary form, gives the name of the telemetry command sent

- LENGTH: Sent in binary form, gives the length in bytes of the telemetry command
- DATA: Sent in binary form, gives just the data of the telemetry command
- ASCII: Data sent in readable form, with a ready bit and timestamp before the actual data

See Table 4 and Table 5 for all possible telemetry commands to the Supervisor and GPS, respectively.

Table 4: All possible SupMCU telemetry commands

Source: [8]

SupMCU Telemetry

Commands	Parameters		
	Name	Length [bytes]	Format
SUP:TEL? 0, [param]	Firmware Version	48	string
SUP:TEL? 1, [param]	SCPI Commands Parsed	8	U ₆₄
SUP:TEL? 2, [param]	SCPI Errors (<i>SCPI commands rejected</i>)	8	U ₆₄
SUP:TEL? 4, [param]	SupMCU CPU Selftests	22	U ₆₄ , U ₆₄ , i ₁₆ , i ₁₆ , i ₁₆
SUP:TEL? 5, [param]	Elapsed time in seconds	8	U ₆₄
SUP:TEL? 6, [param]	Context Switching Occurred	8	U ₆₄
SUP:TEL? 7, [param]	Idling Hooks Remaining	8	U ₆₄
SUP:TEL? 8, [param]	MCU Load	4	f ₃₂
SUP:TEL? 9, [param]	Serial Number	2	U ₁₆
SUP:TEL? 10,[param]	I ² C Address	1	U ₈
SUP:TEL? 11,[param]	Oscillator Tuning Value	1	i ₈
SUP:TEL? 12,[param]	NVM Write Cycles	2	U ₁₆
SUP:TEL? 13, [param]	PIC24 RCON reset cause	2	U ₁₆
SUP:TEL? 14, [param]	Number of Telemetry Items [SupMCU, Module]	4	U ₁₆ , U ₁₆
SUP:TEL? 15, [param]	SupMCU (CTMU) Temperature [0.1K]	2	U ₁₆

Table 5: All possible GPS telemetry commands

Source: [8]

GPSRM Telemetry

Commands	Parameters		
	Name	Length [bytes]	Format
GPS:TEL? 0, [param]	Power Status	2	hex
GPS:TEL? 1, [param]	NMEA String (not functional) ⁶	512	Ascii
GPS:TEL? 2, [param]	Orbit Propagator Results	56	$f_{64}, f_{64}, f_{64}, f_{64}, f_{64}, f_{64},$ f_{64}
GPS:TEL? 3, [param]	Novatel OEM615 Power [mW]	8	$u_{16}, u_{16}, u_{16}, u_{16}$

Power Status (Bit-field)	
State values pertaining to GPS power, reset, position valid, and pass through.	
Bit	Explanation
0	1 = Power is applied to OEM615/719
1	1 = RESET signal is applied to OEM615/719
2	1 = Position Valid pin on OEM615/719 is HIGH (ON)
3	1 = Pass through is enabled on GPSRM.

5.2 Message Handling

Message handling for the GPS commands is similar to EPS commands. To send the message, we first create the vector that holds the message and pass the reference into the `i2c_write` function, returning the transfer status. These functions are simple because non-telemetry GPS and Supervisor messages do not return a value. Depending on the data, the vector is copied into an array for returned telemetry messages and then joined into an integer or a string.

6 Testing

Once preliminary function development was complete, extensive testing was carried out to ensure the functionality of the communication protocol itself, along with the subsystem functionality that was developed. The tools used for testing are the Saleae Logic Pro 16 logic analyzer and CuteCom, a graphical serial terminal. The logic analyzer decodes signals from the I2C lines to confirm that bytes are being transferred and acknowledged by the target and controller. CuteCom was used to verify the success of tests and print the output as data to be interpreted.

The test setup for the EPS involves powering it using a power supply connected to the Electrical Ground Support Equipment (EGSE), which then connects to the EPS. With jumpers, the EPS's SDA, SCL, and GND pins are connected to the Zedboard, which is powered by its own supply. The Zedboard is connected to a Linux PC, which flashes and runs the test using Vitis. The logic analyzer taps into the SDA and SCL jumpers and connects to the PC. See Figure 5 for the EPS test configuration.



Figure 5: ZedBoard (left) connected to EPS (middle), powered by EGSE (right)

Source: Primary

The test setup for the GPS is similar, except that it can be powered via a USB connection to the PC, as seen in Figure 6.

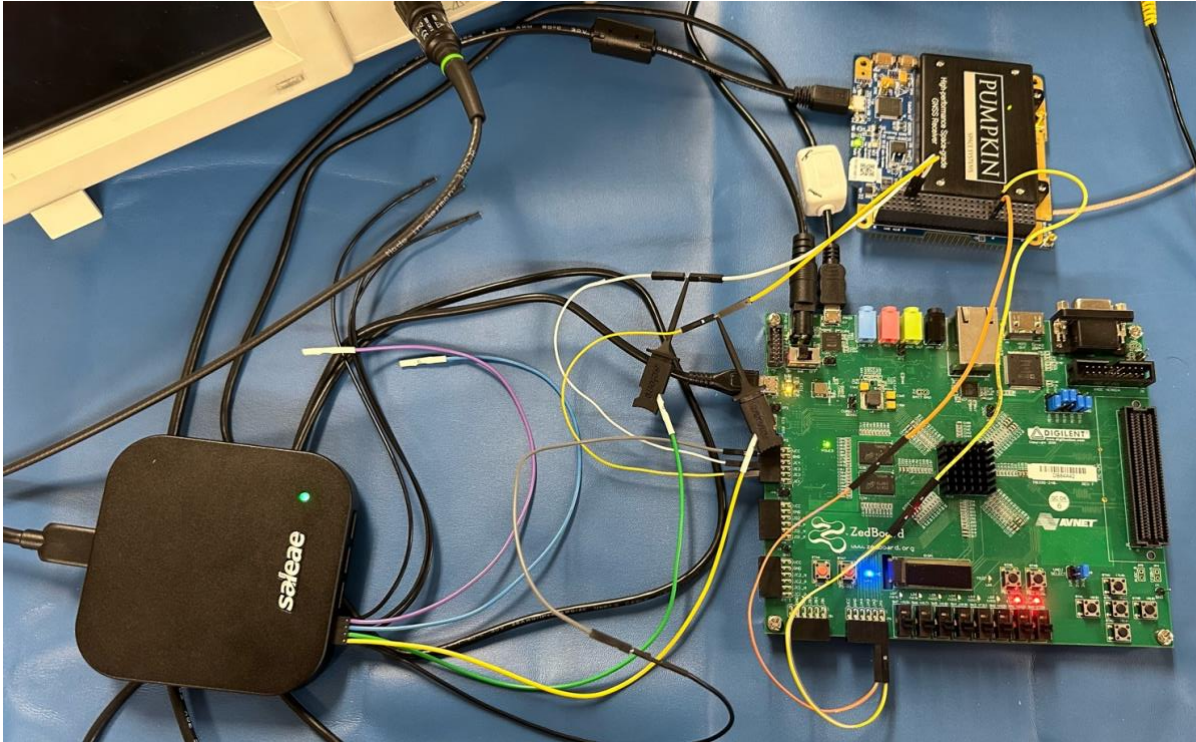


Figure 6: Logic analyzer (left) tapping I2C lines connecting ZedBoard and GPS

Source: Primary

6.1 Logic Analyzer Testing

Before ensuring that the correct bytes were transferred over I2C, the logic analyzer confirmed that the I2C signal exists and is being acknowledged. A simple test was performed on the GPS, sending the command “SUP:LED ON\n” which, if received, turns the SupMCU LED on. After tapping into the lines and opening up the Logic 2 software, a trigger was set up along with I2C decoding, which allows the analyzer to read and convert the I2C signal to the corresponding byte. The test confirmed that the GPS

acknowledged the address transmitted from the OBC (Figure 7), and the SupMCU light was also turned on, further confirming success (Figure 8).

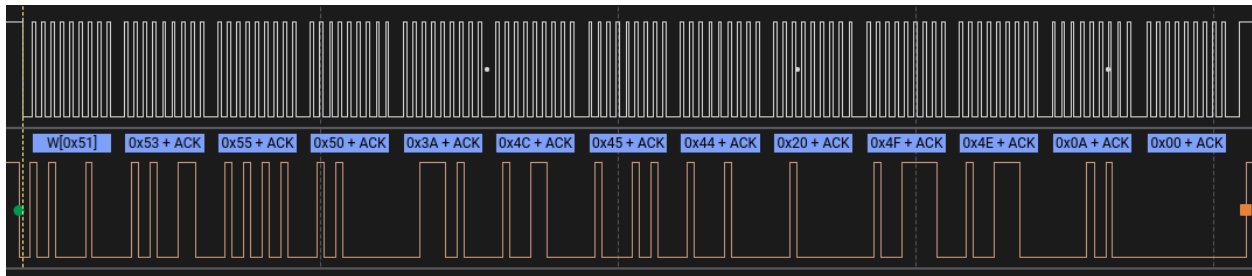


Figure 7: SCL and SDA lines recorded in Logic 2 software

Source: Primary



Figure 8: SupMCU LED on after command

Source: Primary

The tests with the logic analyzer proved that the HAL-level functions are correctly initializing and are compatible with the Vivado hardware design, meaning everything is configured correctly at a low level.

6.2 CuteCom Testing

CuteCom testing involved developing test functions for the command functions created in the EPS and GPS modules. A status value would be returned, and for the functions that returned telemetry data, this data would also be printed to the terminal to confirm it was recorded and formatted correctly. Opening the CuteCom terminal is straightforward. Once the ZedBoard is powered on, we can access the serial connection to the PC via USB. Anything printed in the code when run on the ZedBoard will be output in the CuteCom terminal.

Using CuteCom, it was confirmed that telemetry outputs from both the GPS and EPS were being received correctly. Figure 9 shows the firmware version from the “SUP:TEL? 0,ASCII\n” command in the terminal. In Figure 10, we see all the Get System Status command values as the output after calling the test function, which prints out all the values saved in the struct mentioned in the EPS Implementation section. Getting the correct output from these tests means that the subsystem modules are recording the received data correctly for use by the client.

```
[12:04:44:950] Starting I2C Write and Read Test  
[12:04:44:973] Read Buffer: [1:297047] GPSRM Rev D fw v1.3.5a & SupMCU Core v2.0.0a<break>  
[12:04:44:979]  
[12:04:44:979] Test Passed
```

Figure 9: Output of GPS "SUP:TEL? 0,ASCII\n" in CuteCom

Source: Primary

In Figure 10, we see all the Get System Status command values as the output after calling the test function, which prints out all the values saved in the struct mentioned in the EPS Implementation section. Note that the UNIX time has not yet been set, but is

still formatted correctly. Getting the correct output from these tests means that the subsystem modules are recording the received data correctly for use by the client.

```
[14:33:23:368] Starting EPS get sysstat test c_R l_F
[14:33:23:409] c_R l_F
[14:33:23:411] MODE = 1 c_R l_F
[14:33:23:411] CONF = 0 c_R l_F
[14:33:23:412] RESET_CAUSE = 0 c_R l_F
[14:33:23:414] UPTIME = 126 c_R l_F
[14:33:23:414] ERROR = 0 c_R l_F
[14:33:23:415] RC_CNT_PWRON = 482 c_R l_F
[14:33:23:417] RC_CNT_WDG = 72 c_R l_F
[14:33:23:419] RC_CNT_CMD = 13 c_R l_F
[14:33:23:421] RC_CNT_MCU = 0 c_R l_F
[14:33:23:424] RC_CNT_EMLOPO = 1 c_R l_F
[14:33:23:424] PREVCMD_ELAPSED = 121 c_R l_F
[14:33:23:426] UNIX_TIME = 946695012 c_R l_F
[14:33:23:430] UNIX_YEAR = 0 c_R l_F
[14:33:23:430] UNIX_MONTH = 1 c_R l_F
[14:33:23:430] UNIX_DAY = 1 c_R l_F
[14:33:23:432] UNIX_HOUR = 2 c_R l_F
[14:33:23:432] UNIX_MINUTE = 50 c_R l_F
[14:33:23:435] UNIX_SECOND = 12 c_R l_F
[14:33:23:437] SUCCESS c_R l_F
```

Figure 10: Output of the EPS Get System Status command in CuteCom

Source: Primary

7 Conclusion

Laying the foundation for satellite communication will allow for more streamlined development of satellite systems in the future. Communication between the GPS, EPS, and OBC will allow for more advanced development of the overarching state machines that handle the processes across the satellite.

Through the configuration of solid HAL functions, using the effective tools of the Xilinx Vivado suite, I2C functionality has been streamlined to a level above that of the base Xilinx drivers. Providing tailored error handling in line with the rest of the MUNSTAR-1 protocols and abstracting unnecessary information not needed by the client, robust read and write functions have been created for I2C communication.

On top of the HAL functions, successful implementations of both EPS and GPS functionality were created. Through referencing the message formats in the documentation of both subsystems, functions were created using the I2C protocol to communicate with the EPS and GPS. Using vectors to efficiently pass the command messages into functions and store data that were read, it was confirmed through rigorous testing that complete control over the subsystems could be had using I2C.

With I2C functionality confirmed, this paves the way for developing the Nanomind OBC and complete system testing as future goals. As MUNSTAR-1 moves closer to its launch, the advancements made on the communication protocols serve as a landmark in development as it now shifts towards state machine development.

8 References

- [1] Canadian Space Agency, "About CUBICS," [Online]. Available: <https://www.asc-csa.gc.ca/eng/satellites/cubics/about.asp>. [Accessed 18 June 2024].
- [2] IONOS, "What is a wrapper in programming?," 14 September 2020. [Online]. Available: <https://www.ionos.ca/digitalguide/websites/web-development/what-is-a-wrapper/>. [Accessed 6 August 2024].
- [3] J. Wu, "A Basic Guide to I2C," Texas Instruments, November 2022. [Online]. Available: <https://www.ti.com/lit/an/sbaa565/sbaa565.pdf>. [Accessed 5 August 2024].
- [4] sparkfun, "I2C," [Online]. Available: <https://learn.sparkfun.com/tutorials/i2c/all>. [Accessed 5 August 2024].
- [5] Advanced Micro Devices, "iicps - Vitis Drivers API Documentation," [Online]. Available: <https://xilinx.github.io/embeddedsw.github.io/iicps/doc/html/api/globals.html>. [Accessed 7 August 2024].
- [6] L. Rotthier, V. Timofeyev and M. Kidron, *ISIS Electrical Power System 2 - Software ICD - IVID 7*, ISISPACE, 2020.
- [7] PUMPKIN Space Systems, "UM-10 User Manual GPSRM 1 GPS Receiver Module," 12 June 2020. [Online]. Available: https://www.pumpkininc.com/space/manual/UM-10_Pumpkin_GPSRM_1.pdf. [Accessed 8 August 2024].
- [8] PUMPKIN Space Systems, "SUPERNOVA Firmware Reference Manual Rev 4.4," 2019. [Online]. Available: https://www.pumpkininc.com/space/SupMCU/SFR_Manual_4.4.pdf. [Accessed 8 August 2024].